



Platform Sharing Architecture

🕒 24 minute read

📅 Last updated January 2024

Introduction

The Salesforce sharing model is an essential element in your organization's ability to provide secure application data access. Therefore, it's crucial to architect your sharing model correctly to meet your current and future data access requirements. In this document, we'll review data accessibility components, sharing model use cases, and real customer sharing solutions, and we'll provide some troubleshooting guidelines.

Intended Audience and Prerequisites

This document is intended for advanced system administrators and architects. To understand the concepts, you must have a working knowledge of the Salesforce security and sharing model. Prerequisites to this guide are:

- [Salesforce Help: Sharing Settings](#) 
- [Trailhead Module: Data Security](#) 
- [Designing Record Access for Enterprise Scale](#) 
- [Salesforce Well-Architected - Secure](#)

Data Access Not Covered

This guide focuses on the main features used to control record-level access to standard and custom objects. Topics not covered in this guide include:

- Folder access
- Content access
- Chatter access
- Knowledge Base access
- Ideas, Questions/Answers access
- Mobile data accessibility

Types Of Data Access

Record-level security lets you give users access to some object records, but not others. As with most applications, data access begins with a user. The application must know who the user is before it provides access. For Salesforce, there are different types of users and, sometimes, the level of access is different by type. Instead of reviewing every attribute of every license type, we'll focus here on the interesting attributes that have significant impact on data access. **Record ownership and full access are synonymous and interchangeable and provide the user with the highest level of access to a record.**

Users and Licenses

High-Volume Users

High-volume users (including users with the Customer Community, High Volume Customer Portal, and Authenticated Website license types) don't utilize the sharing model, because their license types don't support roles. These licenses have their own sharing model that works by foreign key match between the user (holding the license) and the data on Account and Contact lookups. Admins can create sharing sets or share groups to grant high-volume users access to records.

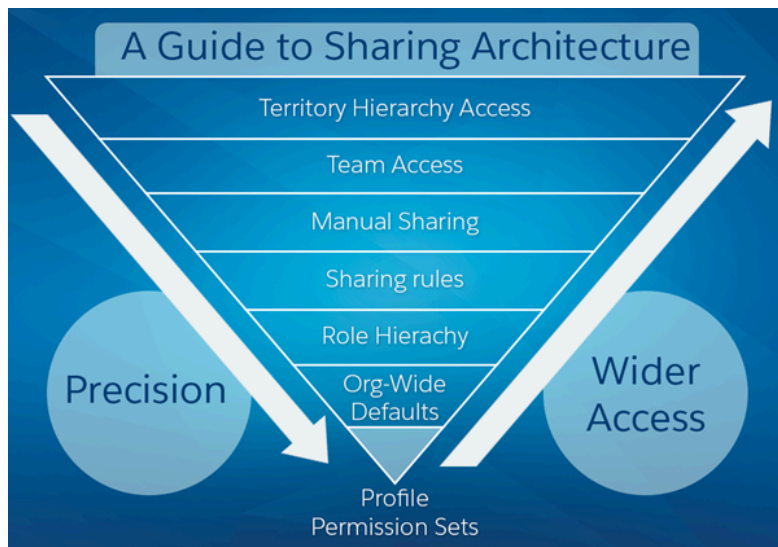
Chatter Free and Chatter External Licenses



The Chatter Free and Chatter External licenses don't follow the standard sharing model. These licenses don't have access to CRM records (standard or custom objects) and Content functionality, and don't support roles, therefore, there is no sharing.

For the remainder of this document, we are assuming a Salesforce user type utilizing a full sharing model. See [User Licenses](#) for more information about each available license type.

Components



Profiles and Permission Sets

Profiles and permission sets provide object-level security by determining what types of data users see and whether they can edit, create, or delete records. For each object, the "View All" and "Modify All" permissions ignore sharing rules and settings, allowing administrators to quickly grant access to records associated with a given object across the organization. These permissions are often preferable alternatives to the "View All Data" and "Modify All Data" administrative permissions, which allow users to view or modify all apps and data.

Profiles and permission sets also control field-level security, which determines the fields within every object that users can access. For example, an object can have 20 fields, but field-level security can be set up to prevent the users from seeing five of the 20 fields.

We strongly recommend that you use permission sets and permission set groups instead of profiles to manage your users' object permissions. Because you can reuse smaller permission set building blocks, you can avoid creating dozens or even hundreds of profiles for each user and job function.

Record Ownership and Queues

Every record must be owned by a single user or a queue. The owner has access to the record, based on the Object Settings for the owner's profile. For example, if the owner's profile has Create and Read permission on an object, but not Edit or Delete permission, the owner can create a record for the object and see the new record. However, the owner won't be able to edit or delete the record. Users higher in a hierarchy (role or territory) inherit the same data access as their subordinates for standard objects. If the subordinate has read-only access, so will the users above them in the role hierarchy. This access applies to records owned by users, as well as records shared with them.

Queues help you prioritize, distribute, and assign records to teams who share workloads. Queue members and users higher in the role hierarchy can access queues from list views and take ownership of records in a queue.

If a single user owns more than 10,000 records, as a best practice:

- The user record of the owner should not hold a role in the role hierarchy.
- If the owner's user record must hold a role, put the role at the top of the hierarchy in its own branch of the role hierarchy.

See [Salesforce Well-Architected - Reliable](#) for more information.

Organization-Wide Defaults

Organization-wide sharing settings specify the default level of access that users have to each others' records. You use organization-wide sharing settings to lock your data to the most restrictive level. Use



the other record-level security and sharing tools to selectively give access to other users. For example, users have object-level permissions to read and edit opportunities, and the organization-wide sharing setting is Read-Only. By default, those users can read all opportunity records, but can't edit any unless they own the record or are granted other permissions.

Organization-wide default settings can be changed from one setting to another (Private to Controlled by Parent, then back to Private). However, these changes require sharing recalculation and depending on volume could result in long processing times.

For custom objects only, you can configure the Grant Access Using Hierarchies setting. If unchecked (default is checked), users higher in the role hierarchy are prevented from inheriting access. This setting is found in the organization-wide default settings.

Role Hierarchy

A role hierarchy represents a level of data access that a user or group of users needs. The role hierarchy ensures that managers always have access to the same data as their employees, regardless of the organization-wide default settings. Role hierarchies don't have to match your organization chart exactly. Instead, each role in the hierarchy should represent a level of data access that a user or group of users needs.

In Salesforce orgs created in Spring '21 or later, you can create up to 5,000 roles. In orgs created before Spring '21, you can create up to 500 roles and can contact Salesforce Customer Support to increase this limit. As a best practice, keep the number of internal roles to 25,000 and the number of external roles to 100,000.

As a best practice, keep the role hierarchy to no more than 10 levels of branches in the hierarchy.

When a user's role changes, any relevant sharing rules are evaluated to correct access as necessary. Peers within the same role don't guarantee them access to each other's data.

Modeling the role hierarchy begins with understanding how the organization is structured. This is usually built from understanding a manager's scope, starting from the top. The CEO oversees the entire company. The CEO usually has direct reports that can then be segmented by Business Unit (Sales or Support) or geographical region (EMEA, APAC). That person then has direct reports that could be further segmented, and so on. Although this sounds very much like an HR organizational chart, when modeling data access, focus on data accessibility with a consideration to HR reporting.

Overlays are always the tricky part of the hierarchy. If they're in their own branch, they'll require either sharing rules, teams, or territory management to gain needed access. If they are folded into the hierarchy, there might be reporting implications.

It's important to spend the time setting up the role hierarchy because it's one of the foundational aspects of the sharing model.

Role Hierarchy Use Cases
Management access – the ability for managers to be able to see and do whatever their subordinates can see and do.
Management reporting – the ability for reporting to roll up in a hierarchical fashion so that anyone higher in the hierarchy sees more data than those below them.
Segregation between organizational branches – different business units don't need to see each other's data; having a hierarchy in which you can define separate branches allows you to segregate visibility within business units, while still rolling visibility up to the executive levels above those units.
Segregation within a role – in many organizations/applications, people who all play the same role should not necessarily see each other's data. Having hierarchical roles allows you to define a "leaf" node in which all data is essentially private, and still roll that data up to a parent role that can see all.

Public Groups

A public group is a collection of individual users, roles, territories, and so on, that all have a function in common. Public groups can consist of:

- Users
- Customer Portal Users
- Partner Users
- Roles
- Roles and Internal Subordinates



- Roles, Internal, and Portal Subordinates
- Portal Roles
- Portal Roles and Subordinates
- Territories
- Territories and Subordinates
- Other public groups (nesting)

Groups can be nested (Group A nested into Group B), but don't nest more than five levels. Nesting has an impact on group maintenance and performance due to group membership calculation. As a best practice, keep the total number of public groups for an organization to 100,000.

Public Group Use Cases
When you need to provide access to an arbitrary group of people, you create a public group to collect them, and then use other sharing tools to give the group the necessary access. Group membership alone doesn't provide data access.
Groups can also be nested inside each other, therefore, allowing a nested group to gain the same access as the group in which it is contained. This allows the creation of smaller, ad-hoc hierarchies that don't necessarily interact with the role or territory hierarchies. If Group A is a member of Group B, then the members of Group A will have access to data shared to Group B at the same access level as the members of Group B.
Groups also have the ability to protect data shared in the group from being made accessible to people in the role hierarchy above the group members. This (and dealing with the access of record owners and their management hierarchy) allows the creation of groups in which very highly confidential information can be shared—the data will be accessible ONLY to group members, and nobody else in the organization. This is accomplished by using the Grant Access Using Hierarchies setting.

Owner-Based Sharing Rules

Owner-based sharing rules allow for exceptions to organization-wide default settings and the role hierarchy that give additional users access to records they don't own. Owner-based sharing rules are based on the record owner only.

Contact owner-based sharing rules don't apply to private contacts, or contacts that aren't associated with an account.

The limit of total sharing rules per object is 300.

Owner-Based Sharing Rule Use Cases
Role-based matrix management or overlay situations: a person in Service must be able to see some Sales data, but they live in different branches of the hierarchy, so you can create a rule that shares data between roles on different branches.
To provide data access to peers who hold the same role or territory.
To provide data access to other groupings of users (public groups, portal. roles, territories). The members of the groupings who own the records can be shared with the members of other groupings.

Criteria-Based Sharing Rules

Criteria-based sharing rules provide access to records based on the record's field values (criteria). If the criteria are met (one or many field values), then a share record is created for the rule. Record ownership is not a consideration.

The limit of criteria-based and guest user sharing rules per object is 50.

Criteria-Based Sharing Rule Use Case
To provide data access to users or groups based on the value of a field on the record.

Guest User Sharing Rules

A guest user sharing rule is a special type of criteria-based sharing rule used to grant record access to unauthenticated guest users. The limit of criteria-based and guest user sharing rules per object is 50.



Warning: The guest user sharing rule type grants access to guest users without login credentials. By creating a guest user sharing rule, you're allowing immediate and unlimited access to all records matching the sharing rule's criteria to anyone. To secure your Salesforce data and give your guest users access to what they need, consider all the use cases and implications of creating this type of sharing rule. Implement security controls that you think are appropriate for the sensitivity of your data. Salesforce is not responsible for any exposure of your data to unauthenticated users based on this change from default settings.

Guest User Sharing Rule Use Case

To provide data access to unauthenticated guest users.

Manual Sharing

Sometimes it's impossible to define a consistent group of users who need access to a particular set of records. Record owners or other users with adequate privileges, such as system admins, can use manual sharing to give read and edit permissions to users who don't have access any other way. Manual sharing isn't automated like organization-wide sharing settings, role hierarchies, or sharing rules. But it gives record owners the flexibility to share records with users that must see them.

Manual sharing is removed when the record owner changes or when the sharing access granted doesn't grant additional access beyond the object's organization-wide sharing default access level. This also applies to manual shares created programmatically.

Manual share records are defined as share records with the row cause set to manual share.

All share records (standard and custom objects) with a row cause set to *manual share* can be edited and deleted by the **Share** button on the object's page layout, even if the share record was created programmatically.

Manual Sharing Use Case

To provide the user with the ability to give access (read only or read/write) to the current record to other users, groups, or roles.

Teams

A team is a group of users that work together on an account, sales opportunity, or case. Record owners can build a team for each record that they own. The record owner adds team members and specifies the access level each team member has to the record. Some team members can have read-only access, while others have read/write.

Only owners, people higher in the hierarchy, and administrators can add team members and provide more access to the member. A team member with read/write access can add another member who already has access to the record with which the team is associated. The team member can't provide them additional access.

Creating a team member in the app creates two records: a team record and an associated share record. If you create team members programmatically, you have to manage both the team record and associated share record. There is only one team per record (Account, Opportunity, or Case). If multiple teams are needed, depending on your specific needs, consider territory management or programmatic sharing

Team Use Cases

To provide the user with the ability to give access (read-only or read/write) to a single group of users (the team).

If teams are managed externally, say through an external commission or territory management system, then integration can be used to manage the account team. There are cases when territory management in an external system can align to a team solution within Salesforce.

Multiple owners of an account can be managed by the account team.

A single group of users (team) require either read-only or read/write access to an opportunity record. (Opportunity Team)



Territory Hierarchy

When using Enterprise Territory Management, you set up a territory hierarchy that shows a model's territory structure and serves as its main interaction point. If Enterprise Territory Management is enabled, you must manage both the role hierarchy and territory hierarchy. For more information, see [Enterprise Territory Management](#) in Salesforce Help.

Apex Managed Sharing

Apex managed sharing (also known as programmatic sharing) allows you to use code to build sophisticated and dynamic sharing settings when a data access requirement can't be met by any other means.

If you create a share record programmatically, and the out-of-box row cause (*manual share*) is used, then you can maintain this share record with the **Share** button in the app. The share record is subject to all rules with the manual share row cause such as deletion upon owner transfer.

You can also create your own Apex sharing reasons for custom objects to track why a record with a user or group of users and simplify the coding required to make updates and deletions of sharing records.

For more information, review [Sharing a Record Using Apex](#) before you consider using programmatic sharing.

Apex Managed Sharing Use Cases
No other method of sharing (declarative) meets the data access needs.
There is an existing, external system of truth for user access assignments which will continue to drive access and be integrated with Salesforce.
Poor performance by using native sharing components. (Usually applies to very large data volumes)
Team functionality on custom objects.

Restriction Rules

In your data access configuration, you may want to prevent users from seeing records that can contain sensitive data or simply aren't essential to their work. You can use restriction rules to allow users to access only records meeting the criteria that you specify. When a restriction rule is applied to a user, the records that the user is granted access to via org-wide defaults, sharing rules, and other sharing mechanisms are filtered by criteria that you specify. Restriction rules work in the opposite manner as the sharing components discussed in this topic; instead of opening up access to users, they further restrict it.

Restriction rules are available for custom objects, external objects, contracts, events, tasks, time sheets, and time sheet entries. You can create up to two active restriction rules per object in Enterprise and Developer editions and up to five active restriction rules per object in Performance and Unlimited editions. Restriction rules are applied to the following Salesforce features:

- List views
- Lookups
- Related lists
- Reports
- Search
- SOQL
- SOSL

Restriction Rule Use Cases
You want certain users to see only a specific set of records.
To simplify controlling access to records with sensitive or confidential information.
To make access to contracts, tasks, and event truly private, as this can be difficult to achieve using organization-wide defaults.

Implicit Sharing



Implicit sharing is automatic. You can neither turn it off, nor turn it on—it is native to the application. In other words, implicit sharing isn't a configurable setting; however, it's important for any architect to understand. **Parent implicit sharing is providing access to parent records (account only) when a user has access to children opportunities, cases, or contacts for that account. Salesforce has a data access policy that states if a user can see a contact (or opportunity, case, or order), then the user implicitly sees the associated account. Child implicit sharing is providing access to an account's child records to the account owner. This access is defined at the owner's role in the role hierarchy.** Child implicit sharing only applies to contact, opportunity, and case objects (children of the account). The access levels that can be provided are View, Edit, and No access for each of the children objects when the role is created. By setting to View, the account owner can implicitly see the associated object records (contact, opportunity, or case). By setting to Edit, the account owner can implicitly modify the associated object (contact, opportunity or case).

Implicit sharing doesn't apply to custom objects.

Customer Implementation Scenarios

There isn't a sharing model that fits all Salesforce orgs. Every org has different requirements and challenges when trying to architect the best sharing model. It's crucial to use the most appropriate data access components to fit the sharing requirements of the org. The following scenarios are common challenges when trying to architect a sharing model.

Team Assignment Managed Externally via Customer Master System

Requirements or Challenges	Solution
Two in a box: a sales manager of one geographic coverage area also wants access to another geographic coverage area in order to assist.	Owner-based sharing rule: An owner-based sharing rule works here because these are edge cases and not the norm. It is also acceptable if the owner-based sharing rule provides more access than is truly necessary because this is a manager – a trusted individual.
Country-based operations users need access to all country sales data.	Owner-based sharing rule: A very common use of a sharing rule is when a different department (other than sales) needs access to sales data.
At least 80% of the time, there is a “core 4” team on an account (Account Executive, Inside Sales Rep, Sales Consultant, Technical Sales Rep). The system of record for the account team assignment is external. There is always only one team to an account.	Teams: Since there is always only one team per account, even if there are many different members with different roles, the account team functionality satisfies this requirement.
Managers of the team must have the same access as their subordinates.	Role hierarchy: The role hierarchy solves this requirement by allowing managers to have access to the data of their subordinates.
The assigned account team must not be modifiable.	Teams: This requirement is not actually accomplished with the account team functionality. However, it also shouldn't prevent you from still using account teams. There are multiple ways of locking down the teams, however. For this case, removing the account team page layout is used.
There must be “buddy” functionality so that when someone is sick or on vacation, someone without standard access to an account or opportunity can be accessed and covered during the time off.	Teams: A “buddy” can simply be a role on the team that accomplishes this requirement. However, the challenge comes from the previous requirement where Teams must not be modifiable. The only solution is to have a set group of people who can modify teams to create the buddy role when necessary.
When a deal requires a custom solution, additional people (who are not necessarily in the sales organization) must have access to the deal.	Teams: A pretty standard usage of the Opportunity Team accomplished by manually adding a new member to the Opportunity Team (via related list). Can also be accomplished via a trigger if you always know who should be added. In this case, it is opportunity by opportunity.



Out-of-Box Territory Management

Requirements or Challenges	Solution
Two different opportunity teams from two distinct business units (Retail Sales and Remarketing) need access to the same account record. They must share contacts and be aware of all activities on the account. These two business units have their own hierarchy and rollups.	Territory management: The best way to think of this requirement is having two branches of a hierarchy that may be structured differently. What justifies territory management is that there are two levels of these two different branches (both levels with members = the opportunity team for that business unit) who need access to the account. Although you could have accomplished this requirement with a Teaming concept, that was too granular. The sales segmentation was not at an account level but in a hierarchy.
There is a separate group of business developers who are assigned and need access to specific accounts for a specific opportunity team (a territory). The business developers are shared resources for the opportunity teams, which mean each business developer may be assigned to one or more accounts for one or more opportunity teams.	Territory management: Because this is a group of users (or a team), and each business development team could be different by account, and since territory management was needed for another reason, then the likely best approach is to build out sub-territories or separate branches that represent these business development teams.
There are non-commission based sales supporting roles who need access to accounts on a one-off basis.	Teams: The key portion of the requirement is "one-off basis." This means it is done on an account by account basis so account teams provide that natively.
The credit department needs access to all accounts for a given business unit.	Owner-based sharing rule: This is a situation where across the board, for a given business unit, a group of users must see everything. This requirement could be accomplished with a sharing rule for a role the group belongs to, a branch of the role hierarchy the group belongs to (role and subordinates), or even a public group. Territory management: You could also accomplish this requirement by modeling the credit department as a territory with the same logic for the given business unit.
Managers must have the same access as their subordinates.	Role hierarchy: The role hierarchy solves this requirement by allowing managers to have access to the data of their subordinates.

Considerations

Enterprise Territory Management Implementation Considerations

- What Happens to the Role Hierarchy?
 - Nothing happens to the role hierarchy if you're also using a territory hierarchy. However, if you're using both territory-based forecasting and role-based forecasting together, you must manage two hierarchies. In this case, we recommend that you use the role hierarchy to model the HR reporting structure, then use the territory hierarchy to model the actual sales hierarchy. The territory hierarchy is best for modeling a matrix reporting structure, where someone can report to multiple managers.

If you're not using territory-based forecasting and role-based forecasting together, it's possible to use only the territory hierarchy as the sales hierarchy. In this scenario, it's best to simplify, or flatten, the hierarchy as much as possible.

We don't recommend making the role hierarchy and territory hierarchy identical because it causes unnecessary sharing activity.

- Can You Still Use Teams?
 - Yes. However, if you can satisfy your access requirements within the territory hierarchy (like overlays), it's better to do it there than to use teams. You're already maintaining multiple



hierarchies (role and territory), so in trying to keep things as simple as possible, only implement teams if no other existing sharing component will satisfy the requirement.

Other Considerations

- Realignment and Reassignment
 - There are two types of changes that occur – the membership of roles, teams, or territories, and the structure of the hierarchy. Membership changes can typically occur daily, even hourly. Hierarchy structural (realignment) changes generally occur less often (quarterly, semi-annually, or annually) and can be resource expensive. What must be considered are the volume of changes and the number of cascading changes that each change will cause. As a rule of thumb, have structural changes occur no more than quarterly and all changes of high volume (bulk or mass changes) be well planned, tested, and coordinated.

- Large Data Volumes

- Whether you're modeling for the initial rollout or planning a realignment change, you must make some serious consideration to the volume of data. There are thresholds where performance can become a factor, so testing out your changes in a sandbox is highly recommended before production. This testing will also give you a baseline for how long the change will take.

If you have more than two million accounts, and have implemented teams or Enterprise Territory Management, you especially must pay attention to performance. These are complex sharing model components that can make for a huge volume of share records and hence, long running transactions.

- Defer Sharing Calculations

- If you have an object that utilizes sharing and has a large volume of records (such as more than two million accounts), and you must make a bulk change (such as a quarterly realignment requiring a hierarchy change), then there's a feature that Salesforce Customer Support can enable to defer automatic sharing calculations. Natively, every individual change to the role hierarchy, territory hierarchy, groups, sharing rules, user roles, team membership, or ownership of records can initiate automatic sharing calculations. When a bulk change is made, it causes a number of automatic sharing recalculations to begin. By suspending these recalculations temporarily, you are able to make the change and then have sharing calculations happen all at once. This method is typically more efficient and better performing for bulk changes.

- Data and Ownership Skews

- Data skews are defined as a few parent records with many children records. These skews can really hurt you when a few accounts have many contacts, opportunities, or cases. The ratio where we start seeing performance degradation is 1:10,000. As a best practice, keep the ratio as close to that as possible (lower is preferred).

Ownership skews are similar to data skews, except we're referring to a single user, role, or group owning many records for an object. As with data skews, these can also cause long running transactions, causing a performance degradation when change occurs. The recommended ratio of owner to number of records is also 1:10,000.

If a single user owns more than 10,000 records, as a best practice:

- The user record of the owner should not hold a role in the role hierarchy.
- If the owner's user record must hold a role, put the role at the top of the hierarchy in its own branch of the role hierarchy.

- Account Hierarchies' Impact on Data Access

- A lot of people make a bad assumption when they implement an account hierarchy. They assume the users who can access a parent account can also access the children accounts. The simple fact of only having a parent/child relationship between two records doesn't drive access. Although the role hierarchy and the territory hierarchy do work in this way, the account hierarchy doesn't.

Troubleshooting



Once you have completed your sharing model architecture, you will likely be challenged with why a user can or can't see a record. Typically, you won't hear when users can see something they shouldn't, but if necessary, there is a way to see every user who has access to a record and why.

The more difficult challenge, and probably more common, is why a user can't see a record. The security layers that you have architected determine where you start. If you know the sharing model well, then you will probably know what component should have provided the access and can start there. But if you are less familiar with the sharing model, start with the role hierarchy and peel back each layer to determine which one should provide the access. Here is a troubleshooting flow.

1. Verify that the user has permissions to access the object.
2. Identify the user's role who can't see the record and note it.
3. Identify the owner of the record's role and note it.
4. Review the role hierarchy and verify that these two roles are in two different branches (they should be).
5. Now you must review the sharing rules for the object and make sure that there is no rule that grants the user access. If needed, look in public groups as well. Maybe the user was left out of a group where there is a sharing rule, or it makes sense to create a new sharing rule to grant the user access? This choice depends on the architecture you are trying to maintain, and applies to both owner-based sharing rules and criteria-based sharing rules.
6. If you're using teams, should this user be on the team for that record? How are teams maintained and how did the miss occur?
7. If manual sharing is used, the user may have lost access because the record owner changed. Manual shares are dropped when ownership changes. The manual share could also have been removed using the **Share** button.
8. If you're using Enterprise Territory Management, is the user missing from one of the territories? Where is the membership of territories maintained and how did the miss occur? Or, maybe the record didn't get assigned to the territory where the user is a member.
9. If you're creating programmatic shares and there are criteria for creating the share in code, review the code to understand why this user was omitted.

Introduction

Intended Audience and Prerequisites

Data Access Not Covered

Types Of Data Access

Users and Licenses

Components

Customer Implementation Scenarios

Team Assignment Managed Externally via Customer Master System

Out-of-Box Territory Management

Considerations

Enterprise Territory Management Implementation Considerations

Other Considerations

Troubleshooting

Well-Architected	Diagrams	Decision Guides	Fundamentals	Resources	Connect
Overview	Salesforce Diagrams Overview	Event-Driven Architectures	Platform Transformation	Project Resource Gallery	Our Blog
Trusted			Architecture Basics	Roadmap Explorer	Our Youtube Channel
Easy	Reference Architecture Gallery	Data Integration	Platform Multitenant Architecture	Release Overviews	Trailblazer Community
Adaptable	Data Model Gallery	Record-Triggered Automation			
Pattern & Anti-Pattern Explorer		Building Forms	Plaform Sharing Architecture		
		See All Guides			

